

```

1411         "end_s": return_s,
1412         "payload_kg": 0.0,
1413         **back,
1414     }
1415 )
1416 energy += back["energy_kwh"]
1417 return_soc = 1.0 - energy / g.energy_kwh
1418 if return_soc < g.reserve - 1e-9:
1419     return None
1420 return {
1421     "start_s": start_s,
1422     "takeoff_s": takeoff_s,
1423     "return_s": return_s,
1424     "duration_s": return_s - start_s,
1425     "payload_kg": total_mass,
1426     "volume_m3": total_volume,
1427     "energy_kwh": energy,
1428     "return_soc": return_soc,
1429     "completions": completions,
1430     "leg_records": leg_records,
1431     "service_records": service_records,
1432     "stops": stops,
1433 }

```

代码清单4对应正文的两套确定性任务构造。它先形成硬时限任务，按需枚举同目的地软箱补入，再对剩余软箱调用问题一精确组批，并贪心合并能够节能的异区单点任务。

Listing 4 问题二基线与混装任务构造

```

1490 def _build_q2_specs(
1491     data: dict[str, Any],
1492     q1_by_service: dict[str, list[dict[str, Any]]],
1493     hard_assignments: dict[str, str] | None = None,
1494 ) -> list[dict[str, Any]]:
1495     boxes: pd.DataFrame = data["boxes"]
1496     hard_specs: list[dict[str, Any]] = []
1497     soft_specs: list[dict[str, Any]] = []
1498     for service_id in sorted(k for k in data["nodes"] if k.startswith("S")):
1499         service_boxes = boxes[boxes["服务区编号"] == service_id].copy()
1500         service_boxes["_hard"] = service_boxes.apply(box_deadline, axis=1)
1501         hard_ids = service_boxes.loc[service_boxes["_hard"].notna(), "货箱编号"].tolist()
1502         soft = service_boxes[service_boxes["_hard"].isna()].drop(columns=["_hard"])
1503         if hard_ids:
1504             chosen_soft: list[str] = []
1505             forced_unit = (hard_assignments or {}).get(service_id)
1506             if forced_unit is not None and len(soft):
1507                 type_id = next(
1508                     x["type_id"] for x in data["transport_units"] if x["unit_id"] == forced_unit
1509                 )
1510                 g = data["transport_types"][type_id]
1511                 soft_rows = soft.reset_index(drop=True)
1512                 best: tuple[tuple[float, int, float, float], list[str]] | None = None
1513                 for mask in range(1 << len(soft_rows)):
1514                     extra_ids = [
1515                         str(soft_rows.iloc[i]["货箱编号"])
1516                         for i in range(len(soft_rows)) if mask & (1 << i)
1517                     ]
1518                     ids = hard_ids + extra_ids
1519                     timeline = route_timeline(
1520                         data["dem"], data["nodes"]["001"], data["nodes"], g, boxes,
1521                         [{"service_id": service_id, "box_ids": ids}], 0.0,
1522                     )
1523                     if timeline is None or any(
1524                         not row["hard_on_time"] for row in timeline["completions"]
1525                     ):
1526                         continue
1527                     selected = soft_rows[soft_rows["货箱编号"].astype(str).isin(extra_ids)]
1528                     urgency = float(
1529                         (selected["应急优先系数"] / selected["期望送达时间 (s)"]).sum()
1530                     )
1531                     score = (
1532                         urgency,
1533                         len(extra_ids),
1534                         float(selected["单箱质量 (kg)"].sum()),

```